

33 -- LLM Observability and Tracing

What it is

LLM observability is the practice of recording structured telemetry for every AI call your application makes -- both text-generation (LLM) calls and embedding calls. For each call, you capture:

- **Provider name** -- which service handled the request (Gemini, Groq, OpenRouter, Jina, local BGE).
- **Model** -- the specific model variant (e.g. `gemini-2.5-flash`, `jina-embeddings-v3`).
- **Operation type** -- `generate` (text generation) or `embed` (vector embedding).
- **Token counts** -- how many input tokens went in, how many output tokens came back.
- **Latency** -- how long the call took in milliseconds.
- **Estimated cost** -- a dollar estimate based on per-1K-token pricing.
- **Status and error message** -- whether the call succeeded or failed, and if it failed, why.

All of this goes into a database table (`llm_traces`) so you can query it, aggregate it, and display it on a dashboard. The goal is the same as application performance monitoring (APM) but focused specifically on AI/LLM calls, which are the most expensive and most failure-prone part of an AI-powered pipeline.

Without observability, you are flying blind: you do not know which provider is consuming the most tokens, whether error rates are climbing, or how much your ETL run actually cost.

Real-life analogy

Think of your **phone bill**. Every month it arrives with an itemized list:

Call	Who you called	Duration	Cost
1	Mom (landline)	12 min	\$0.00
2	Support hotline	45 min	\$3.20
3	International call	3 min	\$1.80

You can see at a glance who you called the most, which calls were expensive, how long they lasted, and if any calls failed (dropped). You can also see totals: total minutes, total cost, number of failed calls.

LLM observability is the same thing but for AI calls. Each "call" is an LLM generation or embedding request. The "who you called" is the provider (Gemini, Groq, etc.). The "duration" is latency. The "cost" is estimated from token counts. And "failed calls" are API errors (429 rate limits, timeouts, bad responses).

Without that itemized bill, you would have no idea why your phone bill doubled last month. Without LLM tracing, you have no idea why your ETL run suddenly costs more or takes longer.

Why we used it here

This project has a **multi-provider architecture** mandated by requirements R22 and R23:

- **3 LLM providers** (R22): Gemini, Groq, OpenRouter -- any of which can be selected at runtime via the Admin UI.
- **3 embedding providers** (R23): Gemini, Jina, local BGE-m3 -- also switchable at runtime.

That is 6 different AI backends, each with different pricing, latency characteristics, rate limits, and failure modes. The admin can switch providers at any time via the Admin UI, and the ETL reads the active provider from the `app_config` DB table at the start of each run.

This creates several problems that observability solves:

1. **Which provider actually served each call?** -- When debugging a bad translation, you need to know if it came from Gemini or Groq. The trace log tells you.
 2. **Token budget tracking** -- Free tiers have limits (Gemini, Groq). If you burn through tokens on one provider, you need to see that before you hit the wall.
 3. **Latency comparison** -- Groq is fast, local BGE has no network overhead, Gemini has variable latency. The dashboard shows avg and max latency per provider so you can make informed switching decisions.
 4. **Error rate monitoring** -- 429 rate-limit errors are common on free tiers. The error count per provider tells you when you are hitting limits.
 5. **UAT evidence** -- The `EVIDENCE_LLM_TRACE` and `EVIDENCE_EMBED_TRACE` log markers prove that every call is tracked, satisfying the client's audit requirement.
-

Code walkthrough

The observability system has four layers: the trace recorder, the provider wrappers, the API endpoint, and the admin UI page.

Layer 1: The trace recorder -- `search_api/services/llm_traces.py`

This file is the core of the system. It defines the database table, the recording function, the wrapper functions, and the aggregation query.

The `llm_traces` table schema:

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS llm_traces (
        id SERIAL PRIMARY KEY,
        provider TEXT NOT NULL,
        model TEXT NOT NULL,
        operation TEXT NOT NULL,          -- 'generate' or 'embed'
        input_tokens INT DEFAULT 0,
        output_tokens INT DEFAULT 0,
        latency_ms FLOAT NOT NULL,
        estimated_cost FLOAT DEFAULT 0.0,
        status TEXT DEFAULT 'success',    -- 'success' or 'error'
        error_message TEXT,
        metadata JSONB DEFAULT '{}',      -- extra info (e.g. text_count)
        created_at TIMESTAMPTZ DEFAULT NOW()
    )
""")
```

The table is created lazily on first use (`_ensure_table()`) with a double-checked lock pattern (`_TABLE_ENSURED` flag + threading lock) so it only runs once per process. Two indexes are created: one on `created_at` DESC (for the "recent traces" query) and one on `provider` (for the "by provider" grouping).

The `record_trace()` function:

```
def record_trace(
    provider: str,
    model: str,
    operation: str,
    input_tokens: int,
    output_tokens: int,
    latency_ms: float,
    status: str = "success",
    error_message: str | None = None,
```

```

        metadata: dict | None = None,
    ) -> None:

```

This is the low-level insert. It looks up the provider in `COST_PER_1K_TOKENS` to calculate `estimated_cost`, then inserts one row. The cost map currently has all zeros (free tiers), but the structure is ready for when paid tiers are used -- you just fill in the rates.

Key design choice: `record_trace()` wraps the entire insert in a try/except and logs a warning on failure instead of raising. Tracing must never break the actual LLM call -- observability is best-effort.

Token estimation:

```

def estimate_tokens(text: str) -> int:
    return max(1, len(text) // 4)

```

A rough heuristic: 1 token per 4 characters. Not exact, but close enough for monitoring purposes. The alternative (calling a tokenizer) would add latency to every traced call.

The `traced_llm_generate()` wrapper:

```

def traced_llm_generate(llm, prompt: str, system_prompt: str = "") -> str:
    input_text = (system_prompt + "\n" + prompt) if system_prompt else prompt
    input_tokens = _estimate_tokens(input_text)

    start = time.perf_counter()
    try:
        result = llm.generate(prompt, system_prompt=system_prompt)
        elapsed = (time.perf_counter() - start) * 1000
        output_tokens = _estimate_tokens(result)
        record_trace(
            provider=llm.name,
            model=getattr(llm, "model", "unknown"),
            operation="generate",
            input_tokens=input_tokens,
            output_tokens=output_tokens,
            latency_ms=elapsed,
        )
        logger.info(
            f"EVIDENCE LLM TRACE: provider={llm.name} op=generate "
            f"in={input_tokens}tok out={output_tokens}tok latency={elapsed}ms"
        )
    except Exception as e:
        elapsed = (time.perf_counter() - start) * 1000
        record_trace(
            provider=llm.name,

```

```

        model=getattr(llm, "model", "unknown"),
        operation="generate",
        input_tokens=input_tokens,
        output_tokens=0,
        latency_ms=elapsed,
        status="error",
        error_message=str(e)[:500],
    )
    raise

```

This wraps any `LLMProvider.generate()` call. It measures wall-clock time with `time.perf_counter()`, estimates tokens for both input and output, records the trace, and logs the `EVIDENCE LLM TRACE` marker. On error, it still records the trace (with `status="error"` and the truncated error message) before re-raising the exception. This means failed calls are visible in the dashboard too.

The `traced_embed()` wrapper:

Same pattern as `traced_llm_generate()` but for embedding calls. It records `text_count` and `dimensions` in the metadata JSONB column, and logs `EVIDENCE_EMBED_TRACE`.

The `get_observability_stats()` aggregation:

```
def get_observability_stats(hours: int = 24) -> dict:
```

This runs three queries against the `llm_traces` table, all filtered to a time window:

1. **Summary query** -- `COUNT(*)`, `SUM(input_tokens)`, `SUM(output_tokens)`, `SUM(estimated_cost)`, `AVG(latency_ms)`, and `COUNT(*) FILTER (WHERE status = 'error')`. This feeds the stat cards.
2. **By-provider query** -- `GROUP BY provider, model, operation` with the same aggregates plus `MAX(latency_ms)`. This feeds the "Usage by Provider" table.
3. **Recent traces query** -- The 50 most recent individual trace rows. This feeds the "Recent Traces" table.

The `FILTER (WHERE ...)` clause is a PostgreSQL feature that lets you compute conditional aggregates in a single pass -- the error count without a separate query.

Layer 2: Provider integration

LLM provider -- `bilingual_etl/providers/llm_provider.py` :

The hook is inside `generate_with_retry()`, the function that every caller in the ETL pipeline uses:

```
def generate_with_retry(llm: LLMProvider, prompt: str, system_prompt: str):
    # ...
    for attempt in range(1, GENERATE_MAX_RETRIES + 1):
        try:
            try:
                from search_api.services.llm_traces import traced_llm
                return traced_llm.generate(llm, prompt, system_prompt=system_prompt)
            except ImportError:
                return llm.generate(prompt, system_prompt=system_prompt)
        except Exception as e:
            # retry logic...
```

The import is inside a `try/except ImportError`. This means tracing is automatically active when the `search_api` package is available (normal Docker Compose setup), but the ETL can still run standalone without the `search_api` installed. The tracing is transparent -- callers of `generate_with_retry()` do not know or care that their calls are being traced.

Embedding provider -- `bilingual_etl/providers/embedding_provider.py` :

The `TracedEmbeddingProvider` wrapper class uses the decorator pattern:

```
class TracedEmbeddingProvider(EmbeddingProvider):
    def __init__(self, inner: EmbeddingProvider):
        self._inner = inner
        self.name = inner.name
        self.dimensions = inner.dimensions

    def embed(self, texts: list[str]) -> list[list[float]]:
        try:
            from search_api.services.llm_traces import traced_embed
            return traced_embed(self._inner, texts)
        except ImportError:
            return self._inner.embed(texts)
```

And `get_embedding_provider()` always wraps the raw provider in this class:

```
def get_embedding_provider(provider_name: str | None = None) -> EmbeddingProvider:
    # ...
    return TracedEmbeddingProvider(cls())
```

Every embedding provider is automatically traced. The wrapper delegates to `traced_embed()`, which calls `self._inner.embed(texts)` internally, measures the time, and records the trace. Same graceful fallback on `ImportError`.

Layer 3: The API endpoint -- `search_api/routers/observability.py`

A single GET endpoint that returns the aggregated stats:

```
@router.get("/admin/observability")
def get_observability(hours: int = Query(24, ge=1, le=720)):
    logger.info(f"EVIDENCE OBSERVABILITY QUERY: hours={hours}")
    return get_observability_stats(hours=hours)
```

The `hours` parameter defaults to 24 and caps at 720 (30 days). The response is a JSON object with three keys: `summary`, `by_provider`, and `recent_traces`.

Layer 4: The Admin UI page -- `admin-ui/src/pages/Observability.jsx`

The React component renders three sections:

1. **Stat cards** (6 cards in a grid) -- Total Calls, Input Tokens, Output Tokens, Avg Latency, Errors (red if > 0, green if 0), Estimated Cost. Numbers use a `formatNumber()` helper that abbreviates large values (e.g. 1500 becomes "1.5K").
2. **Usage by Provider table** -- One row per provider/model/operation combination. Columns: Provider, Model, Operation (with color-coded badges: blue for "generate", green for "embed"), Calls, Input Tokens, Output Tokens, Avg Latency, Max Latency, Errors.
3. **Recent Traces table** -- The last 50 individual calls. Columns: Time, Provider + model, Operation, In Tokens, Out Tokens, Latency, Status (green "ok" or red "error" badge with tooltip showing the error message).

A dropdown lets the user select the time window (1h, 6h, 24h, 3 days, 7 days) before clicking "Load Traces" to fetch the data.

How to explain this in an interview / to a teammate

"We have six different AI backends in this project -- three for text generation and three for embeddings -- and the admin can switch between them at runtime. To keep track of what is happening, we built

an observability layer that automatically traces every AI call. Each trace records which provider handled the call, how many tokens it consumed, how long it took, whether it succeeded or failed, and an estimated cost. All of that goes into a Postgres table, and the admin dashboard aggregates it into stat cards and tables so you can see at a glance which provider is being used the most, whether error rates are rising, and how much each ETL run costs in tokens. The tracing hooks are transparent -- existing code calls `generate_with_retry()` or `provider.embed()` as before, and the wrapper records the trace without the caller knowing."